

ZyraTech Hub

Backend API Documentation

Complete API Reference & Integration Guide for Backend Team

VERSION

v2.2

DATE

March 5, 2026

TOTAL MODULES

25

BASE URL

api.zyratechhub.com

AUTH

Bearer Token

CONFIDENTIAL — INTERNAL USE ONLY

ZyraTech Hub — Backend API Documentation v2.1

Prepared for Backend Team Meeting — March 5, 2026

Overview

ZyraTech Hub is a **STEM Education Platform** built with a React frontend + Redux state management. The frontend expects a **RESTful API** with **Bearer token authentication**.

This document is the **complete, authoritative backend API spec** — covering all 25 modules the frontend requires.

Base URL: `https://api.zyratechhub.com` (or configured via environment variable) **Content-Type:** `application/json` (except file uploads: `multipart/form-data`)

Standard Response Format

All API responses **must** follow this structure:

```
[
  {
    "success": true | false,
    "data": { ... } | [ ... ],
    "message": "string (for errors or success messages)",
    "meta": {
      "page": 1,
      "limit": 20,
      "total": 100,
      "totalPages": 5
    }
  }
]
```

Error responses should also include:

```
[
  {
    "success": false,
    "message": "Human-readable error message",
    "errors": [
      { "field": "email", "message": "Email is required" }
    ]
  }
]
```

System Roles & Department-Based Security

ZyraTech Hub uses a strict **2-role system** for the admin panel, combined with **department-based boundaries**.
(Note: "Students" and "Partners" interact *ONLY* with public endpoints. They do not possess dashboard roles or access.)

Role	Access Level
super_admin	Unlimited System Access. Can view and edit all sections, manage users, review KYC, process refunds, edit system settings, and view system health.
admin	Department-Restricted Access. An admin can only manage the specific department they are assigned to (e.g., an admin assigned to "Blog Articles" cannot edit "Training Courses" or view "Revenue").

Role-Based Access Control (RBAC) Matrix

Backend Implementation Logic:

1. Check if the user is a `super_admin`. If yes, allow the request unconditionally.
2. If the user is an `admin`, verify their assigned `department` matches the resource being accessed.
3. If the department does not match, return `403 Forbidden` ("You do not have permission to access this department").

Feature Area / Department	Super Admin	Regular Admin
Dashboard Access	✓ Full View	✓ Basic View Only
Training Courses	✓ Edit	✓ Edit ONLY IF dept = "Training Courses"
Blog Articles	✓ Edit	✓ Edit ONLY IF dept = "Blog Articles"
Job Listings	✓ Edit	✓ Edit ONLY IF dept = "Job Listings"
Gallery	✓ Edit	✓ Edit ONLY IF dept = "Gallery"
Projects	✓ Edit	✓ Edit ONLY IF dept = "Projects"
FAQ	✓ Edit	✓ Edit ONLY IF dept = "FAQ"
Testimonials	✓ Edit	✓ Edit ONLY IF dept = "Testimonials"
Team Members	✓ Edit	✓ Edit ONLY IF dept = "Team Members"
Content (CMS)	✓ Edit	✓ Edit ONLY IF assigned relevant dept
Payments	✓ Edit / Refund	✓ View ONLY IF dept = "Payments"
Enrollments	✓ Edit	✓ Edit ONLY IF dept = "Enrollments"
Messages	✓ Edit	✓ Edit ONLY IF dept = "Messages"
Partnerships	✓ Edit	✓ Edit ONLY IF dept = "Partnerships"
Contact Inquiries	✓ Edit	✓ Edit ONLY IF dept = "Contact Inquiries"
Impact Stories	✓ Edit	✓ Edit ONLY IF dept = "Impact Stories"
Newsletter	✓ Edit	✓ Edit ONLY IF dept = "Newsletter"
User Management	✓ All operations	✗ Denied
System Settings & Keys	✓ Configuration	✗ Denied
Activity Logs & Health	✓ View	✗ Denied

Security Requirements

Requirement	Details
Authentication	Bearer token in <code>Authorization</code> header
Department RBAC	Middleware must verify the user's role and department exactly as described above
Rate limiting	Especially for auth endpoints (e.g., 5 login attempts / 15 min)
File uploads	Validate image types (JPEG, PNG, WebP), size limits (5MB images, 10MB docs), store in cloud (Cloudinary / S3)
Payment webhooks	Verify Paystack signature on callback
CORS	Allow frontend domain(s) only
Input validation	Sanitize all inputs, prevent XSS and SQL injection
Password hashing	bcrypt with salt rounds ≥ 12

MODULE 1: Authentication & Onboarding

Base Path: `/auth`

IMPORTANT — Admin Onboarding Flow: The frontend implements a **3-step login flow** for admin accounts:

1. **Login** → authenticate with email/password
2. **Force Password Change** → if `mustChangePassword` is `true` (new accounts created by Super Admin get a temporary password)
3. **KYC Verification** → if `kycStatus` is `not_submitted` or `rejected`, the user is prompted to upload identity documents. If `pending`, they see a status message and can skip to dashboard.

Only after all checks pass does the user reach the admin dashboard.

Core Auth Endpoints:

Method	Endpoint	Description	Auth Required	Request Body	Response
POST	<code>/auth/register</code>	Student self-registration	✗	<code>{ name, email, password, phone }</code>	<code>{ token, refreshToken, user }</code>
POST	<code>/auth/verify-email</code>	Verify email with token	✗	<code>{ token }</code>	<code>{ message }</code>
POST	<code>/auth/login</code>	User login	✗	<code>{ email, password }</code>	<code>{ token, refreshToken, user }</code>
POST	<code>/auth/logout</code>	User logout	✓	—	<code>{ success: true }</code>
POST	<code>/auth/refresh</code>	Refresh token	✗	<code>{ refreshToken }</code>	<code>{ token, refreshToken }</code>
GET	<code>/auth/me</code>	Get current user	✓	—	<code>{ user }</code>
POST	<code>/auth/forgot-password</code>	Request password reset	✗	<code>{ email }</code>	<code>{ message }</code>
POST	<code>/auth/reset-password</code>	Reset password	✗	<code>{ token, newPassword }</code>	<code>{ message }</code>

Onboarding Endpoints (Post-Login Steps):

Method	Endpoint	Description	Auth Required	Request Body	Response
POST	<code>/auth/change-password</code>	Force change temporary password (Step 2)	✓	<code>{ currentPassword, newPassword }</code>	<code>{ user }</code>
POST	<code>/auth/kyc/submit</code>	Submit KYC documents (Step 3)	✓	<code>multipart/form-data</code> — see below	<code>{ user }</code>
GET	<code>/auth/kyc/status</code>	Check current KYC status	✓	—	<code>{ kycStatus, submittedAt, reviewedAt, rejectionReason }</code>

KYC Submission (multipart/form-data):

Field	Type	Required	Description
<code>governmentId</code>	File (image/pdf)	✓	Ghana Card, Passport, Voter ID, or Driver's License
<code>proofOfAddress</code>	File (image/pdf)	✓	Utility bill, bank statement, or official letter

Login Response (note KYC and password fields):

The `/auth/login` response **must** include these fields so the frontend can determine which onboarding step to show:

```
[
  "token": "string",
  "refreshToken": "string",
  "user": {
    "id": "uuid",
    "name": "string",
    "email": "string",
    "role": "super_admin|admin",
    "avatar": "string (url)",
    "phone": "string",
    "department": "string",
    "accountStatus": "active|deactivated|pending_password",
    "mustChangePassword": "boolean",
    "kycStatus": "not_submitted|pending|verified|rejected",
    "createdAt": "timestamp",
    "lastLogin": "timestamp"
  }
]
```

Login Flow Decision Tree (Backend Logic):

```
1. Validate credentials → if invalid → 401 "Invalid email or password"
2. Check accountStatus → if "deactivated" → 403 "Account deactivated, contact Super Admin"
3. Return user with mustChangePassword and kycStatus fields
4. Frontend handles the step routing:
   - mustChangePassword=true → show password change form
   - kycStatus="not_submitted" or "rejected" → show KYC upload form
   - kycStatus="pending" → show KYC under review message (can skip to dashboard)
   - All clear → redirect to dashboard
```

Password Validation Rules:

- Minimum 8 characters
- At least 1 uppercase letter
- At least 1 number
- At least 1 special character (!@#\$\$%^&*)
- Must be different from current password

```
|
---
# MODULE 2: Admin Profile Management

**Base Path:** `/auth/profile`

> *The frontend has a dedicated Admin Profile page with profile editing, password changes, notification pre

| Method | Endpoint | Description | Access |
|-----|-----|-----|-----|
| GET | `/auth/profile` | Get current admin's full profile | Any authenticated |
| PUT | `/auth/profile` | Update profile (name, avatar, phone, title, bio) | Any authenticated |
| PUT | `/auth/profile/password` | Change password | Any authenticated |
| PUT | `/auth/profile/notifications` | Update notification preferences | Any authenticated |
| GET | `/auth/profile/sessions` | List active sessions | Any authenticated |
| DELETE | `/auth/profile/sessions/:id` | Revoke a session | Any authenticated |

### Change Password Request:
```json
{
 "currentPassword": "string",
 "newPassword": "string",
 "confirmPassword": "string"
}
```

Notification Preferences:

```
{
 "emailNotifications": true,
 "pushNotifications": false,
 "newsletters": false,
 "securityAlerts": true,
 "systemUpdates": true
}
```

# MODULE 3: User Management & KYC Administration (Super Admin Only)

Base Path: `/admin/users`

The Super Admin creates admin accounts with temporary passwords. New admins must change their password on first login and submit KYC documents (government ID + proof of address). The Super Admin reviews and approves/rejects KYC submissions. Accounts are **deactivated** (never deleted) to preserve audit trails.

User CRUD Endpoints:

Method	Endpoint	Description	Access
GET	/admin/users	List all users (with pagination, filter, search)	Super Admin
GET	/admin/users/:id	Get single user	Super Admin
POST	/admin/users	Create new user (admin/editor accounts with temp password)	Super Admin
PUT	/admin/users/:id	Update user	Super Admin
PATCH	/admin/users/:id/role	Change user role	Super Admin
PATCH	/admin/users/:id/deactivate	Deactivate user (revoke access)	Super Admin
PATCH	/admin/users/:id/reactivate	Reactivate user (restore access)	Super Admin

KYC Review Endpoints (Super Admin):

Method	Endpoint	Description	Access
GET	/admin/users/:id/kyc	Get user's KYC documents & status	Super Admin
PATCH	/admin/users/:id/kyc/approve	Approve KYC → set <code>kycStatus</code> to <code>verified</code>	Super Admin
PATCH	/admin/users/:id/kyc/reject	Reject KYC → set <code>kycStatus</code> to <code>rejected</code>	Super Admin

Reject KYC Request:

```
[{"reason": "string (e.g. 'Document illegible, please resubmit a clear photo')"}]
```

### KYC Documents Response (GET /admin/users/:id/kyc):

```
[
 {
 "kycStatus": "not_submitted|pending|verified|rejected",
 "documents": {
 "governmentId": {
 "url": "string (file url)",
 "filename": "string",
 "uploadedAt": "timestamp"
 },
 "proofOfAddress": {
 "url": "string (file url)",
 "filename": "string",
 "uploadedAt": "timestamp"
 }
 },
 "submittedAt": "timestamp | null",
 "reviewedAt": "timestamp | null",
 "reviewedBy": "uuid | null",
 "rejectionReason": "string | null"
 }
]
```

### Create Admin Request (POST /admin/users):

When the Super Admin creates a new admin account, the backend should:

1. Generate a **temporary password** (or accept one from Super Admin)
2. Set `mustChangePassword: true`
3. Set `accountStatus: "pending_password"`
4. Set `kycStatus: "not_submitted"`
5. Send a **welcome email** with login credentials

```
[
 {
 "name": "string",
 "email": "string",
 "phone": "string",
 "role": "admin",
 "department": "string",
 "temporaryPassword": "string (optional, auto-generated if not provided)"
 }
]
```

User Model (Full — with KYC fields):

```
[
 "id": "uuid",
 "name": "string",
 "email": "string",
 "role": "super_admin|admin",
 "avatar": "string (url)",
 "phone": "string",
 "department": "string",
 "accountStatus": "active|deactivated|pending_password",
 "mustChangePassword": "boolean",
 "kycStatus": "not_submitted|pending|verified|rejected",
 "kycSubmittedAt": "timestamp | null",
 "kycReviewedAt": "timestamp | null",
 "kycRejectionReason": "string | null",
 "createdAt": "timestamp",
 "lastLogin": "timestamp | null",
 "deactivatedAt": "timestamp | null",
 "deactivationReason": "string | null"
]
```

Query Parameters (GET /admin/users):

Param	Type	Description
page	number	Page number (default: 1)
limit	number	Items per page (default: 20)
search	string	Search by name or email
role	string	Filter by role
accountStatus	string	active , deactivated , pending_password
kycStatus	string	not_submitted , pending , verified , rejected
department	string	Filter by department
sortBy	string	Sort field
sortOrder	string	asc or desc

MODULE 4: File Upload / Media

Base Path: /uploads

Multiple modules require file uploads: job applications (resume), gallery (images), blog (featured images), user avatars, course images, partnership logos. All file uploads should go through this centralized service.

Method	Endpoint	Description	Auth Required
POST	/uploads/image	Upload single image (returns URL)	✓
POST	/uploads/document	Upload document (PDF, DOCX)	✓ or Public (for job applications)
POST	/uploads/bulk	Upload multiple files	✓
DELETE	/uploads/:id	Delete uploaded file	✓ Admin

Upload Response:

```
[{"success": true, "data": {"id": "uuid", "url": "https://res.cloudinary.com/...", "filename": "resume_john.pdf", "mimetype": "application/pdf", "size": 245000, "uploadedAt": "timestamp"}}
```

Constraints:

File Type	Max Size	Allowed Formats
Images	5 MB	JPEG, PNG, WebP, SVG
Documents	10 MB	PDF, DOCX, DOC
Bulk	25 MB total	Mixed

**Recommendation:** Use **Cloudinary** for images (with auto-resize/optimize) and **AWS S3** for documents.

MODULE 5: Training & Courses

Public Base Path: /training-courses    Admin Base Path: /admin/training-courses

Method	Endpoint	Access	Description
GET	/training-courses	Public	List all published courses (with filters)
GET	/training-courses/:slug	Public	Get single course by slug
GET	/admin/training-courses	Admin	List all courses (draft + published)
POST	/admin/training-courses	Admin	Create new course
PUT	/admin/training-courses/:id	Admin	Update course
DELETE	/admin/training-courses/:id	Admin	Delete course
PATCH	/admin/training-courses/:id/publish	Admin	Publish/unpublish course
GET	/admin/training-courses/:id/enrollments	Admin	Get course enrollments

Course Model:

```
[
 {
 "id": "uuid",
 "title": "string",
 "slug": "string",
 "description": "string (rich text)",
 "shortDescription": "string",
 "duration": "string",
 "price": "number",
 "discountPrice": "number | null",
 "category": "string",
 "level": "beginner|intermediate|advanced|professional",
 "image": "string (url)",
 "curriculum": [
 {
 "title": "string",
 "topics": ["array of strings"],
 "duration": "string"
 }
],
 "instructors": [
 {
 "name": "string",
 "title": "string",
 "avatar": "string (url)"
 }
],
 "tools": ["array of strings"],
 "prerequisites": ["array of strings"],
 "outcomes": ["array of strings"],
 "status": "draft|published",
 "featured": "boolean",
 "enrollmentCount": "number",
 "rating": "number",
 "createdAt": "timestamp",
 "updatedAt": "timestamp"
 }
]
```

## Query Parameters (GET /training-courses):

Param	Type	Description
page	number	Page number
limit	number	Items per page
category	string	Filter by category
level	string	Filter by level
search	string	Search by title/description
featured	boolean	Featured courses only
sortBy	string	price , title , createdAt , rating

## MODULE 6: Enrollments

Admin Base Path: /admin/enrollments Student Base Path: /enrollments

## Admin Endpoints:

Method	Endpoint	Description	Access
GET	/admin/enrollments	List all enrollments	Admin
POST	/admin/enrollments	Create enrollment manually	Admin
GET	/admin/enrollments/:id	Get enrollment details	Admin
PATCH	/admin/enrollments/:id/status	Update enrollment status	Admin

## Student Endpoints:

Method	Endpoint	Description	Access
POST	/enrollments	Student self-enroll in course	Student
GET	/enrollments/mine	Get student's own enrollments	Student
GET	/enrollments/mine/:id	Get student's enrollment detail	Student

Enrollment Model:

```
[
 "id": "uuid",
 "studentId": "uuid",
 "studentName": "string",
 "studentEmail": "string",
 "courseId": "uuid",
 "courseTitle": "string",
 "status": "pending|active|completed|cancelled",
 "paymentStatus": "pending|paid|refunded",
 "paymentReference": "string",
 "enrolledAt": "timestamp",
 "completedAt": "timestamp | null",
 "progress": "number (0-100)"
]
```

MODULE 7: Payment System (Paystack Integration)

Customer Base Path: /payments Admin Base Path: /admin/transactions

Customer Endpoints:

Method	Endpoint	Description	Auth
POST	/payments/initialize	Initialize payment	Student
GET	/payments/verify/:reference	Verify payment status	Student
POST	/payments/webhook	Paystack webhook callback	✗ (verify signature)

Admin Endpoints:

Method	Endpoint	Description	Access
GET	/admin/transactions	List all transactions	Admin
GET	/admin/transactions/:id	Get transaction details	Admin
POST	/admin/transactions/:id/refund	Process refund	Super Admin
GET	/admin/invoices	List invoices	Admin
GET	/admin/invoices/:id/download	Download invoice PDF	Admin
GET	/admin/revenue-stats	Revenue analytics	Super Admin

### Transaction Model:

```
{
 "id": "uuid",
 "reference": "string",
 "amount": "number",
 "currency": "GHS",
 "status": "pending|success|failed|refunded",
 "customer": {
 "name": "string",
 "email": "string",
 "phone": "string"
 },
 "courseId": "uuid",
 "courseTitle": "string",
 "paymentMethod": "string",
 "paidAt": "timestamp",
 "refundedAt": "timestamp | null",
 "metadata": "object"
}
```

### Initialize Payment Request:

```
{
 "email": "string",
 "amount": "number (in pesewas)",
 "courseId": "uuid",
 "callbackUrl": "string"
}
```

### Webhook Security:

- Verify `x-paystack-signature` header using HMAC SHA512
- Validate event type is `charge.success`
- Verify amount matches expected course price
- Use idempotency to prevent double-processing

## MODULE 8: Content Management (CMS)

**Base Path:** `/admin/content`

*This is the centralized content management system for all static/marketing pages. The frontend has dedicated admin pages for each section.*

## Services (Our Services page)

Method	Endpoint	Description
GET	/admin/content/services	Get all services
POST	/admin/content/services	Create service
PUT	/admin/content/services/:id	Update service
DELETE	/admin/content/services/:id	Delete service

## Benefits (Why Choose Us section)

Method	Endpoint	Description
GET	/admin/content/benefits	Get all benefits
POST	/admin/content/benefits	Create benefit
PUT	/admin/content/benefits/:id	Update benefit
DELETE	/admin/content/benefits/:id	Delete benefit

## About Page Sections

Method	Endpoint	Description
GET	/admin/content/about-hero	Get about page header
PUT	/admin/content/about-hero	Update about page header
GET	/admin/content/about-quote	Get mission statement
PUT	/admin/content/about-quote	Update mission statement
GET	/admin/content/about-mission	Get mission section
PUT	/admin/content/about-mission	Update mission section
GET	/admin/content/about-why-ghana	Get Why Ghana section
PUT	/admin/content/about-why-ghana	Update Why Ghana section
GET/POST/PUT/DELETE	/admin/content/milestones	CRUD for company achievements
GET/POST/PUT/DELETE	/admin/content/timeline	CRUD for company history

## Partnership Page Sections

Method	Endpoint	Description
GET	<code>/admin/content/partnership-hero</code>	Get partnership page header
PUT	<code>/admin/content/partnership-hero</code>	Update partnership page header
GET	<code>/admin/content/why-partner</code>	Get Why Partner section
PUT	<code>/admin/content/why-partner</code>	Update Why Partner section
GET/POST/PUT/DELETE	<code>/admin/content/partnership-stories</code>	CRUD for success stories
GET	<code>/admin/content/impact-stats</code>	Get impact stats
PUT	<code>/admin/content/impact-stats/:id</code>	Update impact stat
GET/POST/DELETE	<code>/admin/content/recognition</code>	CRUD for awards/logos

## Work With Us Page

Method	Endpoint	Description
GET	<code>/admin/content/work-with-us</code>	Get all Work With Us content
PUT	<code>/admin/content/work-with-us</code>	Update section (hero, process, collaboration, hiring)

**Work With Us sections** (pass `section` in body):

- `hero` — Hero title, subtitle, description
- `process` — Project setup steps
- `collaboration` — Intercultural collaboration content
- `hiring` — How we hire steps

## Quality Assurance Page

Method	Endpoint	Description
GET	<code>/admin/content/quality-assurance</code>	Get all QA content
PUT	<code>/admin/content/quality-assurance</code>	Update section

**QA sections** (pass `section` in body):

- `hero` — Hero title, description, image
- `intro` — How it works + standards
- `tools` — Technology tools list
- `features` — Feature blocks with content & images
- `faq` — QA-specific FAQ items

Collaboration Models Page

Method	Endpoint	Description
GET	/admin/content/collaboration-models	Get collaboration models content
PUT	/admin/content/collaboration-models	Update collaboration models

MODULE 9: Hero Slider Management

Base Path: /admin/hero-slides

Method	Endpoint	Description
GET	/admin/hero-slides	Get all slides
POST	/admin/hero-slides	Create new slide
PUT	/admin/hero-slides/:id	Update slide
DELETE	/admin/hero-slides/:id	Delete slide
POST	/admin/hero-slides/reorder	Reorder slides

Hero Slide Model:

```
[
 "id": "uuid",
 "title": "string",
 "description": "string",
 "pillar": "string",
 "backgroundImage": "string (url)",
 "cta1Text": "string",
 "cta1Link": "string",
 "cta2Text": "string",
 "cta2Link": "string",
 "isVisible": "boolean",
 "order": "number"
]
```

Reorder Request:

```
[
 "slideIds": ["uuid1", "uuid2", "uuid3"]
]
```

# MODULE 10: Partnerships Management

Public Base Path: `/partnerships` Admin Base Path: `/admin/partnerships`

Method	Endpoint	Access	Description
GET	<code>/partnerships</code>	Public	List all active partnerships
GET	<code>/admin/partnerships</code>	Admin	List all partnerships (with filters)
GET	<code>/admin/partnerships/:id</code>	Admin	Get single partnership
POST	<code>/admin/partnerships</code>	Admin	Create partnership
PUT	<code>/admin/partnerships/:id</code>	Admin	Update partnership
DELETE	<code>/admin/partnerships/:id</code>	Admin	Delete partnership
PATCH	<code>/admin/partnerships/:id/toggle-featured</code>	Admin	Toggle featured status

## Partnership Model:

```
[
 {
 "id": "PART-YYYY-###",
 "organization": {
 "name": "string",
 "logo": "string (url)",
 "website": "string",
 "industry": "string"
 },
 "contact": {
 "name": "string",
 "email": "string",
 "phone": "string",
 "role": "string"
 },
 "type": "corporate|academic|government|technology|ngo",
 "status": "active|negotiating|inactive",
 "featured": "boolean",
 "startDate": "date",
 "endDate": "date | null",
 "value": "string",
 "description": "string",
 "benefits": ["array of strings"],
 "studentsPlaced": "number",
 "projectsCompleted": "number"
 }
]
```

# MODULE 11: Impact Metrics & Success Stories

Base Path: `/api/impact`

Impact Metrics

Method	Endpoint	Access	Description
GET	/api/impact/metrics	Public	Get all active metrics
GET	/api/impact/metrics?location={page}	Public	Get metrics for specific page
GET	/api/impact/metrics/:id	Admin	Get single metric
POST	/api/impact/metrics	Admin	Create metric
PUT	/api/impact/metrics/:id	Admin	Update metric
DELETE	/api/impact/metrics/:id	Admin	Delete metric

Metric Model:

```
[
 {
 "id": "MET-###",
 "title": "string",
 "value": "number|string",
 "previousValue": "number",
 "type": "number|percentage|currency|rating|text",
 "category": "students|employment|partnerships|community|awards|courses|financial|projects|training",
 "description": "string",
 "prefix": "string",
 "suffix": "string",
 "featured": "boolean",
 "active": "boolean",
 "displayOrder": "number",
 "displayLocations": ["home", "partnership", "projects", "training", "about", "impact"],
 "trend": "up|down|stable",
 "lastUpdated": "timestamp"
 }
]
```

Success Stories

Method	Endpoint	Access	Description
GET	/api/impact/stories	Public	Get all published stories
GET	/api/impact/stories/:id	Admin	Get single story
POST	/api/impact/stories	Admin	Create story
PUT	/api/impact/stories/:id	Admin	Update story
DELETE	/api/impact/stories/:id	Admin	Delete story

Story Model:

```
[
 "id": "uuid",
 "title": "string",
 "content": "string (rich text)",
 "image": "string (url)",
 "category": "string",
 "featured": "boolean",
 "status": "published|draft",
 "author": "string",
 "publishedAt": "timestamp",
 "createdAt": "timestamp"
]
```

MODULE 12: Testimonials Management

Public Base Path: /testimonials    Admin Base Path: /admin/testimonials

Method	Endpoint	Access	Description
GET	/testimonials	Public	List published testimonials
GET	/admin/testimonials	Admin	List all testimonials
POST	/admin/testimonials	Admin	Create testimonial
PUT	/admin/testimonials/:id	Admin	Update testimonial
DELETE	/admin/testimonials/:id	Admin	Delete testimonial
PATCH	/admin/testimonials/:id/toggle-featured	Admin	Toggle featured

Testimonial Model:

```
[
 "id": "uuid",
 "name": "string",
 "role": "string",
 "type": "student|alumni|partner|parent|mentor|corporate",
 "quote": "string",
 "rating": "number (1-5)",
 "avatar": "string (url)",
 "featured": "boolean",
 "status": "published|draft|pending",
 "program": "string",
 "date": "date",
 "likes": "number",
 "verified": "boolean"
]
```

# MODULE 13: FAQ Management

Public Base Path: /faqs Admin Base Path: /admin/faqs

Method	Endpoint	Access	Description
GET	/faqs	Public	Get published FAQs only
POST	/faqs/:id/view	Public	Increment view count
POST	/faqs/:id/helpful	Public	Vote FAQ as helpful
GET	/admin/faqs	Admin	Get all FAQs (with drafts)
GET	/admin/faqs/:id	Admin	Get single FAQ
POST	/admin/faqs	Admin	Create FAQ
PUT	/admin/faqs/:id	Admin	Update FAQ
DELETE	/admin/faqs/:id	Admin	Delete FAQ

FAQ Model:

```
[
 {
 "id": "uuid",
 "category": "Internship Program|Services & Support|Partnerships|Donations & Support|Training|General",
 "question": "string",
 "answer": "string (rich text)",
 "status": "published|draft",
 "order": "number",
 "views": "number",
 "helpful": "number",
 "createdAt": "timestamp"
 }
]
```

# MODULE 14: Blog / News System

Public Base Path: /blog-posts Admin Base Path: /admin/blog-posts

Method	Endpoint	Access	Description
GET	/blog-posts	Public	List published posts (paginated)
GET	/blog-posts/:slug	Public	Get single post by slug
POST	/blog-posts/:slug/view	Public	Increment view count
GET	/blog-posts/:slug/related	Public	Get related posts
GET	/blog-posts/categories	Public	List available categories
GET	/admin/blog-posts	Admin	List all posts (draft + published)
POST	/admin/blog-posts	Admin	Create post
PUT	/admin/blog-posts/:id	Admin	Update post
DELETE	/admin/blog-posts/:id	Admin	Delete post
PATCH	/admin/blog-posts/:id/publish	Admin	Publish/unpublish post

Blog Post Model:

```
[
 {
 "id": "uuid",
 "title": "string",
 "slug": "string",
 "excerpt": "string",
 "content": "string (rich text / HTML)",
 "featuredImage": "string (url)",
 "category": "string",
 "tags": ["array of strings"],
 "author": {
 "id": "uuid",
 "name": "string",
 "avatar": "string (url)"
 },
 "status": "draft|published",
 "views": "number",
 "readTime": "string",
 "publishedAt": "timestamp",
 "createdAt": "timestamp",
 "updatedAt": "timestamp"
 }
]
```

MODULE 15: Projects Management

Public Base Path: /projects Admin Base Path: /admin/projects

Method	Endpoint	Access	Description
GET	/projects	Public	List published projects
GET	/projects/:slug	Public	Get project details
GET	/admin/projects	Admin	Admin list (all statuses)
POST	/admin/projects	Admin	Create project
PUT	/admin/projects/:id	Admin	Update project
DELETE	/admin/projects/:id	Admin	Delete project

Project Model:

```
[
 "id": "uuid",
 "title": "string",
 "slug": "string",
 "description": "string (rich text)",
 "shortDescription": "string",
 "image": "string (url)",
 "gallery": ["array of image urls"],
 "category": "string",
 "technologies": ["array of strings"],
 "client": "string",
 "status": "draft|published|completed|in-progress",
 "featured": "boolean",
 "startDate": "date",
 "endDate": "date | null",
 "teamSize": "number",
 "createdAt": "timestamp"
]
```

MODULE 16: Gallery Management

Public Base Path: /gallery Admin Base Path: /admin/gallery

Method	Endpoint	Access	Description
GET	/gallery	Public	List gallery images (by album/category)
GET	/gallery/albums	Public	List albums
POST	/admin/gallery	Admin	Upload image(s)
PUT	/admin/gallery/:id	Admin	Update metadata (caption, album, order)
DELETE	/admin/gallery/:id	Admin	Delete image
POST	/admin/gallery/albums	Admin	Create album
PUT	/admin/gallery/albums/:id	Admin	Update album
DELETE	/admin/gallery/albums/:id	Admin	Delete album

Gallery Image Model:

```
[
 {
 "id": "uuid",
 "url": "string",
 "thumbnail": "string (url)",
 "caption": "string",
 "album": "string",
 "tags": ["array"],
 "order": "number",
 "uploadedAt": "timestamp"
 }
]
```

MODULE 17: Jobs & Applications Management

Public Base Path: /jobs Admin Base Path: /admin/jobs , /admin/job-applications

The frontend has a complete Jobs module: public listings, job detail, application form with resume upload, and full admin CRUD with application review pipeline.

Public Endpoints:

Method	Endpoint	Description
GET	/jobs	List published job listings (with filters)
GET	/jobs/:id	Get single job detail
POST	/jobs/:id/apply	Submit job application (multipart/form-data)

Admin — Job Listings:

Method	Endpoint	Description
GET	/admin/jobs	List all jobs (all statuses)
GET	/admin/jobs/:id	Get single job (admin view with application count)
POST	/admin/jobs	Create job listing
PUT	/admin/jobs/:id	Update job listing
DELETE	/admin/jobs/:id	Delete job listing
PATCH	/admin/jobs/:id/publish	Publish/unpublish/close job
GET	/admin/jobs/:id/applications	Get applications for specific job

Admin — Job Applications:

Method	Endpoint	Description
GET	/admin/job-applications	List all applications (with filters)
GET	/admin/job-applications/:id	Get application details
PATCH	/admin/job-applications/:id/status	Update application status
DELETE	/admin/job-applications/:id	Delete application

Job Model:

```
[
 {
 "id": "uuid",
 "title": "string",
 "slug": "string",
 "type": "Full-time|Part-time|Contract|Internship|National Service|Remote",
 "category": "technical|education|operations",
 "locations": ["Koforidua", "Remote"],
 "description": "string (rich text)",
 "responsibilities": ["array of strings"],
 "qualifications": ["array of strings"],
 "perks": ["array of strings"],
 "salary": {
 "min": "number | null",
 "max": "number | null",
 "currency": "GHS",
 "period": "monthly|annually",
 "display": "string (e.g. 'Competitive')",
 },
 "status": "draft|published|closed|archived",
 "featured": "boolean",
 "applicationDeadline": "date | null",
 "applicationsCount": "number",
 "createdBy": "uuid",
 "createdAt": "timestamp",
 "updatedAt": "timestamp"
 }
]
```

Job Application Model:

```
[
 {
 "id": "uuid",
 "jobId": "uuid",
 "jobTitle": "string",
 "applicant": {
 "firstName": "string",
 "lastName": "string",
 "email": "string",
 "phone": "string",
 "linkedin": "string | null",
 "portfolio": "string | null",
 "location": "string"
 },
 "resume": "string (file url)",
 "coverLetter": "string (file url or text)",
 "experience": [
 {
 "company": "string",
 "title": "string",
 "startDate": "date",
 "endDate": "date | null",
 "current": "boolean",
 "description": "string"
 }
],
 "education": "string",
 "skills": ["array of strings"],
 "availability": "string",
 "expectedSalary": "string | null",
 "howDidYouHear": "string",
 "status": "new|reviewing|shortlisted|interview|offered|hired|rejected",
 "rating": "number (1-5) | null",
 "notes": "string | null",
 "appliedAt": "timestamp",
 "updatedAt": "timestamp"
 }
]
```

Query Parameters (GET /jobs):

Param	Type	Description
page	number	Page number
limit	number	Items per page
category	string	technical , education , operations
type	string	Full-time , Part-time , Contract , etc.
location	string	Filter by location
search	string	Search by title

# MODULE 18: Contact Form & Messages

Public Base Path: `/contact` Admin Base Path: `/admin/messages`

The frontend has a public contact form AND a full admin inbox (927-line `MessagesManagementPage`) with read/unread, starring, replies, archive, bulk actions, categories, and priority levels.

## Public Endpoints:

Method	Endpoint	Description
POST	<code>/contact</code>	Submit contact form (no auth required)

## Contact Form Request:

```
[{"fullName": "string", "email": "string", "phone": "string | null", "inquiryType": "partnership|collaboration|general|support|media", "message": "string", "source": "Contact Form"}]
```

## Admin Endpoints:

Method	Endpoint	Description	Access
GET	<code>/admin/messages</code>	List all messages (with filters)	Admin
GET	<code>/admin/messages/:id</code>	Get single message	Admin
PATCH	<code>/admin/messages/:id/read</code>	Mark as read	Admin
PATCH	<code>/admin/messages/:id/star</code>	Toggle starred	Admin
PATCH	<code>/admin/messages/:id/archive</code>	Archive message	Admin
POST	<code>/admin/messages/:id/reply</code>	Reply to message	Admin
DELETE	<code>/admin/messages/:id</code>	Delete message	Admin
POST	<code>/admin/messages/bulk-action</code>	Bulk action (read, archive, delete)	Admin

Message Model:

```
[
 {
 "id": "MSG-YYYY-###",
 "sender": {
 "name": "string",
 "email": "string",
 "phone": "string | null",
 "company": "string | null"
 },
 "subject": "string",
 "message": "string",
 "category": "general|training|partnership|careers|feedback|support",
 "priority": "high|medium|low",
 "status": "unread|read|replied|archived",
 "starred": "boolean",
 "source": "Contact Form|Email|WhatsApp",
 "replies": [
 {
 "id": "uuid",
 "message": "string",
 "sentBy": {
 "id": "uuid",
 "name": "string"
 },
 "sentAt": "timestamp"
 }
],
 "assignedTo": "uuid | null",
 "createdAt": "timestamp"
 }
]
```

Bulk Action Request:

```
{
 "action": "mark_read|archive|delete",
 "messageIds": ["MSG-2024-001", "MSG-2024-002"]
}
```

Query Parameters (GET /admin/messages):

Param	Type	Description
page	number	Page number
limit	number	Items per page
status	string	unread , read , replied , archived
category	string	Filter by category
priority	string	high , medium , low
starred	boolean	Show starred only
search	string	Search by sender name/email/subject

# MODULE 19: Newsletter Management

Public Base Path: `/newsletter` Admin Base Path: `/admin/newsletter`

The `NewsletterHero` component appears on 13+ public pages and tracks the source page. The admin has a newsletter management page for subscribers and sending.

### Public Endpoints:

Method	Endpoint	Description
POST	<code>/newsletter/subscribe</code>	Subscribe to newsletter (no auth)
POST	<code>/newsletter/unsubscribe</code>	Unsubscribe via email link (token-based)

### Subscribe Request:

```
{
 "email": "string",
 "source": "Homepage|Blog|Training Page|About Page|Partnership Page|Jobs Page|Projects Page|Our Services|Contact Us"
}
```

### Admin Endpoints:

Method	Endpoint	Description	Access
GET	<code>/admin/newsletter/subscribers</code>	List subscribers (with filters)	Admin
GET	<code>/admin/newsletter/subscribers/export</code>	Export subscribers as CSV	Admin
DELETE	<code>/admin/newsletter/subscribers/:id</code>	Remove subscriber	Admin
POST	<code>/admin/newsletter/send</code>	Send newsletter to subscribers	Admin
GET	<code>/admin/newsletter/campaigns</code>	List sent campaigns	Admin
GET	<code>/admin/newsletter/stats</code>	Subscriber stats (growth, sources)	Admin

### Subscriber Model:

```
{
 "id": "uuid",
 "email": "string",
 "source": "string",
 "status": "active|unsubscribed",
 "subscribedAt": "timestamp",
 "unsubscribedAt": "timestamp | null"
}
```

Send Newsletter Request:

```
[
 "subject": "string",
 "body": "string (HTML)",
 "recipients": "all|segment",
 "segmentFilter": {
 "source": "string | null"
 },
 "scheduledAt": "timestamp | null"
]
```

MODULE 20: Activity Logs (Audit Trail)

Base Path: /admin/activity-logs

Method	Endpoint	Description	Access
GET	/admin/activity-logs	Get logs (with filters)	Admin
GET	/admin/activity-logs/stats	Get log statistics	Admin
DELETE	/admin/activity-logs	Clear old logs	Super Admin

Log Entry Model:

```
[
 "id": "LOG-timestamp-random",
 "type": "login|logout|user_created|user_updated|content_updated|payment_received|enrollment_created|course_created",
 "severity": "info|success|warning|error",
 "user": {
 "id": "uuid",
 "name": "string",
 "email": "string",
 "role": "string"
 },
 "description": "string",
 "details": {
 "device": "string",
 "browser": "string",
 "location": "string",
 "ip": "string"
 },
 "resourceType": "string (e.g. 'user', 'course', 'enrollment')",
 "resourceId": "string",
 "timestamp": "timestamp",
 "metadata": "object"
]
```

Query Parameters:

Param	Type	Description
page	number	Page number
limit	number	Items per page
type	string	Filter by activity type
severity	string	Filter by severity
userId	uuid	Filter by user
startDate	date	From date
endDate	date	To date
search	string	Search descriptions

MODULE 21: Site Settings

Admin Base Path: /admin/settings Public Base Path: /settings

Method	Endpoint	Description	Access
GET	/settings/public	Get public settings (branding, contact, social)	Public
GET	/admin/settings	Get all settings	Super Admin
PUT	/admin/settings	Bulk update settings	Super Admin
PUT	/admin/settings/:key	Update single setting	Super Admin

Public Settings Response (non-sensitive only):

```
[
 "siteName": "ZyraTech Hub",
 "siteDescription": "string",
 "tagline": "string",
 "primaryColor": "#004fa2",
 "secondaryColor": "#ff6b35",
 "logoLight": "url | null",
 "logoDark": "url | null",
 "contactEmail": "info@zyratechhub.com",
 "contactPhone": "+233 55 955 4261",
 "contactWhatsApp": "233559554261",
 "contactAddress": "string",
 "socialLinkedIn": "url",
 "socialTwitter": "url",
 "socialInstagram": "url",
 "socialFacebook": "url",
 "socialYouTube": "url",
 "businessHoursWeekday": "8:00 AM - 5:00 PM",
 "businessHoursWeekend": "Closed"
]
```

All Settings Categories (admin only):

Category	Keys
Branding	siteName, siteDescription, tagline, logoLight, logoDark, favicon, primaryColor, secondaryColor
Contact	contactEmail, contactPhone, contactWhatsApp, contactAddress, hrName, hrEmail, hrTitle
Social	socialLinkedIn, socialTwitter, socialInstagram, socialFacebook, socialYouTube
Payment	paystackPublicKey, paystackSecretKey, stripePublicKey, stripeSecretKey, currency
SMTP	smtpHost, smtpPort, smtpUser, smtpPassword, fromEmail, fromName, supportEmail
SEO	seoTitle, seoDescription, seoKeywords, siteUrl
Analytics	googleAnalyticsId, metaPixelId, enableVisitorTracking
General	maintenanceMode, allowRegistration, timezone, businessHoursWeekday, businessHoursWeekend, copyrightText

MODULE 22: Reports & Analytics

Base Path: /admin/reports, /admin/analytics

The frontend has a 1,063-line Reports page and a 644-line Analytics page. The Super Admin dashboard also expects aggregated stats.

Dashboard Stats:

Method	Endpoint	Description	Access
GET	/admin/analytics/dashboard	Aggregated dashboard stats	Admin

Dashboard Response:

```
[
 "totalStudents": "number",
 "totalRevenue": "number",
 "totalEnrollments": "number",
 "totalCourses": "number",
 "activePartners": "number",
 "openJobs": "number",
 "pendingApplications": "number",
 "newsletterSubscribers": "number",
 "unreadMessages": "number",
 "recentActivity": ["array of activity log entries"],
 "trends": {
 "enrollmentGrowth": "percentage",
 "revenueGrowth": "percentage",
 "userGrowth": "percentage"
 }
]
```

Analytics Endpoints:

Method	Endpoint	Description	Access
GET	/admin/analytics/enrollment	Enrollment trends over time	Admin
GET	/admin/analytics/revenue	Revenue breakdown & trends	Super Admin
GET	/admin/analytics/courses	Course performance metrics	Admin
GET	/admin/analytics/users	User growth & activity	Super Admin
GET	/admin/analytics/traffic	Website visitor analytics	Admin

Analytics Query Parameters:

Param	Type	Description
period	string	7d , 30d , 90d , 1y , custom
startDate	date	Custom range start
endDate	date	Custom range end
groupBy	string	day , week , month

Reports CRUD:

Method	Endpoint	Description	Access
GET	/admin/reports	List saved/scheduled reports	Admin
POST	/admin/reports	Create/save a report config	Admin
GET	/admin/reports/:id	Get report details	Admin
PUT	/admin/reports/:id	Update report config	Admin
DELETE	/admin/reports/:id	Delete report	Admin
POST	/admin/reports/:id/run	Generate/run report now	Admin
GET	/admin/reports/:id/download	Download generated report (PDF/CSV/Excel)	Admin
PATCH	/admin/reports/:id/toggle	Pause/activate scheduled report	Admin

Report Model:

```
[
 {
 "id": "RPT-###",
 "name": "string",
 "description": "string",
 "type": "enrollment|revenue|course_performance|partnership|payment|platform_usage|marketing|custom",
 "frequency": "one_time|daily|weekly|monthly|quarterly",
 "status": "active|paused|draft",
 "format": "PDF|Excel|CSV",
 "recipients": ["array of emails"],
 "metrics": ["array of metric names"],
 "dateRange": {
 "start": "date",
 "end": "date"
 },
 "lastGenerated": "timestamp | null",
 "createdBy": "string",
 "createdAt": "timestamp"
 }
]
```

MODULE 23: Notifications (Optional / Phase 2)

Base Path: /admin/notifications

The frontend's Admin Profile page has notification preferences, and dashboards show activity feeds. This module enables real-time-style notifications.

Method	Endpoint	Description	Access
GET	/admin/notifications	List notifications	Authenticated
GET	/admin/notifications/unread-count	Get unread badge count	Authenticated
PATCH	/admin/notifications/:id/read	Mark as read	Authenticated
POST	/admin/notifications/mark-all-read	Mark all as read	Authenticated
DELETE	/admin/notifications/:id	Delete notification	Authenticated

Notification Model:

```
[
 {
 "id": "uuid",
 "type": "enrollment|payment|message|application|system|alert",
 "title": "string",
 "message": "string",
 "link": "string (admin route to navigate to)",
 "read": "boolean",
 "createdAt": "timestamp"
 }
]
```

MODULE 24: Public API Endpoints Summary

Quick reference for all endpoints that do NOT require authentication.

Endpoint	Method	Purpose
/auth/register	POST	Student self-registration
/auth/login	POST	User login
/auth/forgot-password	POST	Password reset request
/auth/reset-password	POST	Reset password
/auth/verify-email	POST	Email verification
/auth/refresh	POST	Refresh token
/settings/public	GET	Public site settings (branding, contact, social)
/training-courses	GET	Published courses list
/training-courses/:slug	GET	Course detail
/jobs	GET	Published job listings
/jobs/:id	GET	Job detail
/jobs/:id/apply	POST	Submit job application
/partnerships	GET	Active partnerships
/testimonials	GET	Published testimonials
/faqs	GET	Published FAQs
/faqs/:id/view	POST	Increment FAQ view count
/faqs/:id/helpful	POST	Vote FAQ as helpful
/blog-posts	GET	Published blog posts
/blog-posts/:slug	GET	Blog post detail
/blog-posts/:slug/view	POST	Increment blog view count
/blog-posts/:slug/related	GET	Related blog posts
/blog-posts/categories	GET	Blog categories
/projects	GET	Published projects
/projects/:slug	GET	Project detail
/gallery	GET	Gallery images
/gallery/albums	GET	Gallery albums
/api/impact/metrics	GET	Impact metrics
/api/impact/stories	GET	Success stories
/contact	POST	Submit contact form
/newsletter/subscribe	POST	Newsletter signup
/newsletter/unsubscribe	POST	Newsletter unsubscribe
/payments/initialize	POST	Initialize payment (requires student auth)
/payments/verify/:ref	GET	Verify payment (requires student auth)

Endpoint	Method	Purpose
/payments/webhook	POST	Paystack webhook (verify signature)

## Implementation Priority

Phase	Modules	Timeline Suggestion
Phase 1 — Foundation	Auth (1,2), User Mgmt (3), File Uploads (4), Settings (21)	Week 1-2
Phase 2 — Core Business	Courses (5), Enrollments (6), Payments (7), Jobs (17)	Week 3-5
Phase 3 — Content	CMS (8), Hero Slider (9), FAQ (13), Contact/Messages (18), Newsletter (19)	Week 6-7
Phase 4 — Marketing	Partnerships (10), Impact (11), Testimonials (12), Blog (14), Projects (15), Gallery (16)	Week 8-9
Phase 5 — Admin Tools	Activity Logs (20), Reports/Analytics (22), Notifications (23)	Week 10-11

## Technical Notes for Backend Team

### Database

- Recommended:** PostgreSQL (relational data, complex queries, full-text search)
- ORM:** Prisma or TypeORM (if using Node.js), or Django ORM (if using Python)

### API Framework

- Node.js:** Express.js or Fastify with validation middleware (Joi/Zod)
- Python:** Django REST Framework or FastAPI

### Authentication Flow

- Login → returns `token` (15min expiry) + `refreshToken` (7 day expiry)
- All requests include `Authorization: Bearer <token>`
- Token expired → frontend calls `/auth/refresh` with `refreshToken`
- Refresh token expired → force logout to login page

### File Storage

- Images:** Cloudinary (auto-resize, CDN, WebP conversion)
- Documents:** AWS S3 or Cloudinary

Email

- **Transactional:** SendGrid, Mailgun, or SMTP
- **Use cases:** Welcome email, password reset, enrollment confirmation, contact form auto-reply, newsletter

Pagination

- Use `page` + `limit` query params
- Always return `meta` object with `page`, `limit`, `total`, `totalPages`

Search

- Implement basic search on relevant text fields
- For full-text search: PostgreSQL `tsvector` or Elasticsearch (later)

Rate Limiting

Endpoint Group	Limit
<code>/auth/login</code>	5 requests / 15 min per IP
<code>/auth/forgot-password</code>	3 requests / hour per email
<code>/contact</code>	5 requests / hour per IP
<code>/newsletter/subscribe</code>	3 requests / hour per IP
General API	100 requests / min per user

MODULE 25: System Health & Monitoring (Super Admin Only)

Base Path: `/admin/system`

The Super Admin Dashboard displays real-time system health metrics including server load, uptime, storage usage, API latency, error rates, active sessions, and security alerts. These need backend endpoints.

Method	Endpoint	Description	Access
GET	<code>/admin/system/health</code>	Get full system health snapshot	Super Admin
GET	<code>/admin/system/storage</code>	Get storage usage stats	Super Admin
GET	<code>/admin/system/sessions</code>	Get active session count	Super Admin
GET	<code>/admin/system/security-alerts</code>	Get security alerts	Super Admin

### System Health Response (GET /admin/system/health):

```
[{"serverLoad": 42, "uptime": 99.9, "storageUsed": 450, "storageTotal": 1000, "apiLatency": 124, "errorRate": 0.02, "activeSessions": 1240, "securityAlerts": 0, "databaseStatus": "healthy", "lastBackup": "timestamp", "checkedAt": "timestamp"}]
```

**Implementation Note:** These can start as simple metrics from the hosting provider's API (e.g., Azure Monitor, Railway, Render). They don't need to be real-time initially — polling every 60 seconds is fine.

## Super Admin — Complete Feature Checklist

Use this checklist to verify that the Super Admin has access to everything. Every item marked  must work for `super_admin` role.

#	Feature	Sidebar Section	Module	Status in Doc
1	<b>Super Admin Dashboard</b> (revenue, system health, quick actions)	Dashboard	22 (Analytics) + 25 (System Health)	 Documented
2	<b>Analytics Page</b> (enrollment trends, revenue, course performance)	Analytics	22	 Documented
3	<b>Hero Slides</b> management	Content	9	 Documented
4	<b>Services</b> management	Content	8 (CMS)	 Documented
5	<b>Benefits / Why Choose Us</b> management	Content	8 (CMS)	 Documented
6	<b>About Section</b> (quote, hero, mission, why ghana, milestones, timeline)	Content	8 (CMS)	 Documented
7	<b>About Page</b> management	Content	8 (CMS)	 Documented
8	<b>Work With Us</b> page management	Content	8 (CMS)	 Documented
9	<b>Quality Assurance</b> page management	Content	8 (CMS)	 Documented
10	<b>Partnership Content</b> management	Content	8 (CMS)	 Documented
11	<b>Training Courses</b> full CRUD + publish	Content	5	 Documented
12	<b>Blog Articles</b> full CRUD + publish	Content	14	 Documented
13	<b>Job Listings</b> full CRUD + applications review	Content	17	 Documented
14	<b>Gallery</b> full CRUD + delete	Content	16	 Documented
15	<b>Projects</b> full CRUD	Content	15	 Documented
16	<b>FAQ</b> full CRUD	Content	13	 Documented
17	<b>Testimonials</b> full CRUD + featured toggle	Content	12	 Documented
18	<b>Payments</b> view + <b>refund</b> (exclusive)	Business	7	 Documented
19	<b>Revenue Stats</b> (exclusive)	Business	7	 Documented
20	<b>Enrollments</b> management	Business	6	 Documented
21	<b>Messages / Inbox</b> full management	Business	18	 Documented

#	Feature	Sidebar Section	Module	Status in Doc
22	<b>Partnership Requests</b> management	Business	10	 Documented
23	<b>Contact Inquiries</b> management	Business	18	 Documented
24	<b>Impact Stories &amp; Metrics</b> management	Business	11	 Documented
25	<b>Newsletter</b> subscribers + send	Business	19	 Documented
26	<b>Administrators</b> (user management, create, deactivate, KYC review) (exclusive)	System	3	 Documented
27	<b>Site Settings</b> (all categories, payment keys) (exclusive for edit)	System	21	 Documented
28	<b>Activity Logs</b> (audit trail) (exclusive)	System	20	 Documented
29	<b>Reports</b> (create, schedule, download) (exclusive)	System	22	 Documented
30	<b>System Health</b> (server, storage, sessions, alerts) (exclusive)	System	25	 Documented
31	<b>Admin Profile</b> (own profile, password, notifications, sessions)	—	2	 Documented
32	<b>KYC Submission</b> (own KYC during onboarding)	—	1	 Documented
33	<b>Notifications</b> (optional / Phase 2)	—	23	 Documented

Document Version: 2.2 | Last Updated: March 5, 2026 | Total Modules: 25 | RBAC Matrix: Complete